

여러 부서가 한 업무 시스템을 함께 쓰는 환경에서 화면을 어디까지 공유하고 어디부터 갈라야 하는지 정하는 일을 세 번 했습니다(05). 자동화를 어디까지 기계에 맡기고 어느 칸을 사람에게 남길지 정하는 일은 지금 하고 있습니다(01). 지원한 자리의 일이 그 둘의 연장선이라고 이해했습니다.

업무를 화면으로 옮기는 게 아니라, 파편화된 프로세스를 다시 구조로 세웁니다.

실무 **4년 2개월**입니다. 그 안에 기획 · IA · 디자인 · 구현 · 배포 · 운영이 분리되지 않은 채로 들어 있습니다. 성격이 다른 두 집단이 한 시스템을 함께 쓰는 구조를 **세 번** 다뤘고, 지금은 Next.js와 TypeScript로 사용자 웹 · 관리자 CMS · API **세 서비스**를 맡고 있습니다.

askedtech.com mstone-demo.vercel.app precisionrct.com

재직 중

개인 작업

단독 리드

01. 지금 하는 일

2026.06 - 2026.08 · 계약직 · IT부서

askedtech · 러닝스파크

에듀테크 정보 플랫폼의 사용자 웹 · 관리자 CMS · API 세 서비스를 담당합니다.

계약 기간이 정해진 자리이고 8월에 종료됩니다. 인계 시점에서 역산해 문서와 자동화를 먼저 남기는 순서로 일하고 있습니다.

문제

자료는 RSS와 메일함으로 나누어 들어오고, 발행까지 섹션 분류 · 요약 · 검수를 거쳐야 합니다.

설계

- RSS와 메일에서 수집해 LLM이 섹션을 분류하고 한국어로 요약합니다. 여기까지는 기계가 합니다.
- 발행 버튼은 사람이 누릅니다. LLM 요약은 대체로 맞지만 가끔 틀리고, 뉴스레터는 회수가 안 됩니다.

결정

- LLM 호출부를 주입 가능한 클라이언트로 분리하고 드라이런 모드를 넣었습니다. 테스트할 때마다 실제 메일이 나가면 안 됩니다.
- 프롬프트보다 먼저 정한 것은 “이 호출을 하지 않고도 전체를 돌려볼 수 있는가”였습니다.

승인 · 사람
되돌릴 수 없음

이 칸을 통과하지 못하면 아무것도 나가지 않습니다.

Next.js · TypeScript · NestJS · MongoDB · 사이트맵 자동화 7,000+ URL

askedtech.com

뉴스레터 파이프라인은 회사 산출물이고 인증 정보가 얹혀 있어 코드와 화면은 공개하지 않습니다. 구조와 판단 근거
까지만 적습니다.

02. 선택한 작업

아래 세 건은 만든 것보다 무엇을 정하고 무엇을 포기했는지를 적었습니다.

2023.06 - 2024.01 · 개발팀 대리

Digital SAT 웹앱 · 교육공간

학원 · 강사 · 학생이 함께 쓰는 B2B 시험 운영 웹앱.

문제

학생 · 시험 · 강의 · 포인트 네 모듈을 만들어야 하는데 화면 정의가 없었습니다.

설계

IA와 와이어프레임을 먼저 만들고 Figma 프로토타입으로 합의한 뒤 React로 구현했습니다. 요구사항을 기다리지 않고, 합의부터 시작했습니다.

결정

- 성적통계 대시보드는 화면보다 지표 정의를 먼저 합의했습니다. 조직 단위와 개인 단위는 축이 달라 같은 자리에 놓으면 둘 다 안 읽힙니다.
- 응시 상태 자동저장 · 복원은 실패 시나리오에서 출발했습니다. 시험 중 브라우저가 닫히면 응시가 사라지므로 저장을 시스템 책임으로 옮겼습니다.

React · Java · MariaDB · Figma - IA · 와이어프레임 · 프로토타입 · UX/UI를 직접 만들고 구현까지 담당

precisionrct.com

2년 5개월간 100% 원격으로 맡은 미국 워싱턴 D.C. 의료기관 사이트. 요구사항 합의부터 인수인계까지 현지 담당자와 영어 서면으로 진행했습니다.

설계

- CMS 없이 PHP + JSON 파일 기반 렌더링 구조를 직접 설계했습니다. WordPress를 없으면 플러그인과 보안 패치 부채를 원격 1인이 영구히 떠안습니다.
- 비개발자가 직접 편집할 수 없다는 비용이 생기는데, 담당자와 합의하고 받아들였습니다.

결정

그 비용은 나갈 때 정산했습니다. 계약 종료 시점에 발행을 자동화해 인계했습니다. 시스템의 성공 여부는 담당자가 바뀔 다음에 드러납니다.

PHP · JSON · jQuery · Bootstrap 5 · SiteGround - 반응형 / 크로스브라우징 · 메타데이터 · 사이트 맵 · OG · 호스팅 · SSL · 백업 · 장애 대응

precisionrct.com

이 사이트의 Lighthouse 점수는 아래 03 표에 낮은 값 그대로 적었습니다.

mStone · 제품 라인업 사이트

기계식 키보드 브랜드의 팬 제작 디자인 시안. 제품 3종의 라인업과 상세.

설계

- 제품 3종을 빌드 시점에 정적으로 만들었습니다. 갱신되지 않는 시안이라 재검증 주기를 정할 근거가 없었습니다.
- 갱신이 생기면 App Router에서 라우트 단위로 켤 수 있는 형태까지만 열어뒀습니다.

결정

데스크톱은 네 항목 모두 100, LCP 0.8초. 같은 페이지 모바일은 성능 76, LCP 4.7초입니다. 데스크톱만 적으면 절반만 말하는 것이라 둘 다 적었습니다.

Next.js · App Router · Vercel - 데스크톱 100 / 100 / 100 / 100 · LCP 0.8s · CLS 0.003
- 모바일 성능 76 · LCP 4.7s

mstone-demo.vercel.app

실존 브랜드와 무관한 팬 제작 시안입니다.

이 밖에 할리스 리뉴얼(프론트엔드 4인 팀 리드 · 듀얼 페르소나 정보구조 · 유료 라이브러리를 대체한 스크롤 인터랙션 자체 구현)과 Apple Pop(디자인 토큰 · i18next 22개 로케일 · iOS / Android 출시)은 jaykim-v.github.io/fe 에 적었습니다.

03. 실측치

잘 나온 것만 심지 않았습니디. 같은 사람이 만든 결과물의 좋은 점수와 나쁜 점수를 같은 표에 넣었습니다. 감이 아니라 수치로 확인합니다.

측정: Chrome Lighthouse (모바일 / 데스크톱 프로파일), 시크릿 창 · 캐시 비활성. — 는 측정하지 않은 항목입니다.

대상	환경	성능	접근성	모범사례	SEO	LCP · CLS
mStone	데스크톱	100	100	100	100	0.8s · 0.003

처음부터 Next.js로 만든 경우.

mStone	모바일	76	100	100	100	4.7s · —
--------	-----	----	-----	-----	-----	----------

데스크톱 0.8초와 갈리는 지점은 회선이 아니라 모바일 프로파일의 CPU 스로틀링입니다. 반복 측정하면 76에서 96까지 흔들리는데, 그 편차가 CPU 민감도의 신호입니다. 다음 작업은 히어로 구간의 렌더 경로를 줄이는 것입니다.

precisionrct.com	데스크톱	67	—	—	—	—
------------------	------	----	---	---	---	---

precisionrct.com	모바일	57	80	100	92	—
------------------	-----	----	----	-----	----	---

jQuery · Bootstrap 5 기반 자산을 2년 5개월 운영한 사이트. 계약 종료 시점의 1순위는 발행 자동화 인계였고 성능은 뒤로 미뤘습니다. 미룬 것도 판단입니다.

jaykim-v.github.io	모바일	55	—	—	—	—
--------------------	-----	----	---	---	---	---

제 기존 포트폴리오입니다. 원인은 CDN 웹폰트 세 벌 · 스크롤 연출 라이브러리 세 개 · 이미지입니다.

이 페이지	데스크톱	100	100	100	100	0.4s · 0
-------	------	-----	-----	-----	-----	----------

이 페이지	모바일	100	100	100	100	1.7s · 0
-------	-----	-----	-----	-----	-----	----------

위 진단을 그대로 적용한 결과입니다. 스크립트 0바이트 · 웹폰트 self-host 서브셋 4파일 104KB · 이미지 0장. CLS 0은 우연이 아니라 폴백 서체의 ascent · descent를 Pretendard 실측값으로 맞춰둔 결과입니다.

가장 낮은 줄은 제 기존 포트폴리오입니다. 이 페이지는 그 세 원인을 각각 self-host 서브셋 · 스크립트 0바이트 · 이미지 0장으로 되돌린 결과입니다.

이력 전체와 상세 규칙은 jaykim-v.github.io/fe 에 있습니다.

04. 토큰과 규칙

규칙을 먼저 정하고 컴포넌트를 엮습니다. 이 페이지의 색 토큰은 새로 만들지 않고 기존 포트폴리오에서 그대로 가져왔습니다. 디자인 시스템의 값은 처음 만들 때보다 다음 화면에서 다시 쓰일 때 생깁니다.

색은 코어 6개로 닫고, 나머지 2개는 같은 계열에서 한 단계 낮은 파생값입니다. 파생값에 새 색을 넣지 않습니다.

간격은 4px 배수만 씁니다. 예외를 한 번 허용하면 다음 화면에서 3px이 생깁니다.

컴포넌트는 색을 받지 않고 의미를 받습니다. 색을 받는 순간 그 컴포넌트가 쓰인 모든 자리가 토큰 교체의 예외가 됩니다.

--bg	--line-soft	--bg-elev	--line	--muted	--ink-soft	--ink	--accent
#FDFCF8	#EFEADC	#F4EFE2	#E5DEC8	#7A7263	#1F2622	#0E1512	#2F5D48

배경 인쇄가 꺼지면 색은 사라지고 값은 남습니다.

접근성은 말이 아니라 대비비로 적습니다. AAA에 못 미치는 조합은 사용 규칙을 함께 적었습니다. 위 값은 sRGB 상대휘도로 계산했습니다.

조합	대비비	판정	사용 규칙
--ink on --bg	18.03:1	AAA	본문 · 제목
--ink-soft on --bg	15.06:1	AAA	본문 보조 · 각주
--accent on --bg	7.36:1	AAA	링크 · 강조
--bg on --accent	7.36:1	AAA	05 밴드의 제목과 본문. 이 면에서는 투명도를 쓰지 않 습니다

05. 일하는 방식

같은 백엔드 위에서 화면을 가릅니다

성격이 다른 두 집단이 한 시스템을 쓰는 구조를 세 번 만났습니다. Digital SAT에서는 학생과 학원 관리자의 정보구조를 직접 설계했고, 할리스 리뉴얼에서는 고객과 창업 예정자의 동선을 팀 리드로서 갈랐고, 교육부 진로멘토링에서는 멘토 포털과 학교 포털이 백엔드를 공유하는 구조에서 화면을 맡았습니다.

어려운 건 화면을 두 벌 만드는 일이 아니라 어디까지 공유하고 어디부터 갈라야 하는지 선을 긋는 일이었습니다. 화면을 나누는 기준이 곧 권한 모델이고, 권한 모델은 나중에 붙일 수 없습니다.

우회를 막는 통제가 아니라, 우회할 이유를 줄이는 설계

승인이 귀찮으면 사람들은 파이프라인을 우회해서 손으로 보냅니다. 그래서 통제를 조이는 대신, 승인 화면을 한 번에 훑고 끝낼 수 있게 만드는 쪽을 먼저 봤습니다. 만들어놓고 안 쓰는 시스템이 아니라 매일 열리는 시스템이 되어야 합니다.

만들지 않기로 한 것

SNS 자동 발행을 검토하면서 개발 공수와 절감 시간을 계산했더니 손익분기가 53주였습니다. 그래서 자동화하지 않았습니다. 대신 사람이 한 번 누르는 반자동 MVP를 제안했고, 설득해서 채택됐습니다. 기술적으로 가능한 것과 지금 만들 것을 구분하는 게 제가 하는 일의 절반입니다.

업무 시스템에 대해 제가 전제하는 것

트래픽은 평균이 아니라 특정 시점에 몰립니다. 그 시점에 배포가 서비스를 멈추면 안 됩니다.

다국어는 나중에 붙이는 기능이 아니라 처음에 정하는 규칙입니다. 22개 로케일을 붙여보고 알았습니다.

자동화의 마지막 한 칸은 사람이 눌러야 합니다. 공개되면 되돌릴 수 없는 것일수록 그렇습니다.

06. 조건과 근거

프론트엔드 채용에서 자주 요구되는 항목에 제 근거를 대응시켰습니다.

각 근거 뒤 괄호는 이 문서의 섹션 번호입니다.

항목	근거	상태
프론트엔드 경력	실무 4년 2개월. 5년 이상을 요구하는 자리라면 10개월이 모자랍니다. 아래 07	4년 2개월
React · Next.js	Digital SAT React 구현(02) · mStone App Router(02) · askedtech 웹 / CMS(01)	실무
TypeScript	askedtech 전 구간(01) · Apple Pop 양대 마켓 출시(02)	실무
REST API · GraphQL 연동	REST: 세 서비스를 잇는 계약 운용(01) · Supabase Edge Functions로 REST / RPC 작성(02). GraphQL: 실무 미적용, 개인 프로젝트에서 스키마 · 리졸버 · DataLoader까지 구현(07)	REST 실무 · GraphQL
웹 성능 최적화 · 접근성	03 실측표에 LCP · CLS 등 Web Vitals 지표를 좋은 값과 나쁜 값 모두 기재 · 04 대비비표(미달 조합의 사용 금지 규칙까지)	실무
협업 주도	기획 · 백엔드와 함께 듀얼 페르소나 구조를 설계(05) · 프론트엔드 4인 팀 리드로 정보구조와 공통 규칙을 먼저 합의(02) · 미국 클라이언트와 2년 5개월 영어 서면 협업(02) · 반자동 MVP를 제안해 설득하고 채택(05)	실무
디자인 · 기획 의도의 기술적 해석	산출물이 없는 상태에서 IA · 와이어프레임 · Figma · UX/UI를 직접 만들고 React로 구현(02) · 토큰을 먼저 정하고 컴포넌트를 엮음(02)	실무

07. 지금 없는 것

GraphQL — 실무 적용 경험 없음

REST로 세 서비스를 잇는 계약을 다루면서, 화면마다 필요한 필드가 달라 엔드포인트가 늘거나 안 쓰는 필드가 쌓이는 문제를 겪었습니다. 그게 GraphQL이 푸는 지점이라는 것까지 확인하고 싶어, 제가 출시한 게임의 Postgres 스키마 모양을 그대로 옮긴 로컬 DB 위에 GraphQL 계층을 직접 엮어봤습니다. 리그 8개 × 25명을 중첩해 조회할 때 쿼리가 409번 나가는 것을 계측기로 확인했고, DataLoader로 배치해 4번으로 줄였습니다. 로더를 요청 단위로 만들어야 한다는 것도 그 과정에서 알았습니다. 다만 운영 규모에서의 캐시 무효화와 스키마 변경 관리는 아직 겪어보지 못했습니다.

github.com/JayKim-v/graphql-n-plus-one

경력 4년 2개월

5년 이상을 요구하는 자리라면 10개월이 모자랍니다. 그 4년 2개월 동안 네 곳에서 일
곱 개 서비스를 맡았고, 그중 2년 5개월은 미국 클라이언트 사이트를 원격으로 혼자 리
드하며 요구사항 합의부터 인수인계까지 영어 서면으로 진행한 기간입니다. 지금도 한
사람이 사용자 웹 · 관리자 CMS · API 세 개를 담당합니다. 기획 · IA · 디자인 · 구현 · 배
포 · 운영이 분리되지 않은 환경에서의 4년 2개월입니다.

09. 연락처

김재희 · jaeheekim.dev@gmail.com · 010-9025-9880

github.com/JayKim-v · 전체 이력 jaykim-v.github.io

이 페이지는 프론트엔드 직무에 맞춰 다시 쓴 버전입니다. 백엔드 · 인프라 · 모바일을 포함한 전체 이력
은 jaykim-v.github.io 에 있고, 그쪽 PDF는 2026년 5월 생성본이라 현재 맡고 있는 일이 반영돼 있
지 않습니다.

스크립트 0바이트 · 이미지 0장 · 웹폰트 self-host 서브셋 4파일

소스: github.com/JayKim-v/JayKim-v.github.io/tree/main/fe